DATA 266 — Lab 1 Report · Team 40

Members: Ruchika Chaurasia · Akhilesh Tandur
Repository: https://github.com/RuchikaChaurasia/DATA266-Lab1-Team40
Kaggle team: PairProgramming_Team_40 · **Date:** October 6, 2026

Ownership statement

- **Ruchika Chaurasia** designed, implemented, trained and evaluated everything in the `ruchika_chaurasia/` folders of all three tasks: the character-level GPT (Task 1), the three Yelp classifiers (Task 2), and the CycleGAN with its main run and seven follow-up experiments (Task 3). She also set up the shared repository structure, the shared evaluation scripts (`task*/common/`), the reproducibility tooling (`common/runtime.py`, raw logs, manifests), the audit tooling and this report. All her runs used an RTX 5090 in the SJSU GPU lab.
- **Akhilesh Tandur** designed, implemented, trained and evaluated everything in the `akhilesh_tandur/` folders of all three tasks, with his own code (`src/gpt_char.py`, `src/sentiment.py`, `src/cyclegan.py`, `src/evaluate_local.py`), also on an RTX 5090.
- Each member trained their own models for every task and submitted their own Kaggle entries for Task 3. Both members reviewed the joint analysis. AI assistance is disclosed in `AI_USE.md`.

Task 1 — GPT-style character-level LLM (TinyStories)

Both models are written from scratch: multi-head causal self-attention, LayerNorm, feed-forward network, residual connections, token and positional embeddings, and an LM head, with no prebuilt Transformer or attention modules. Each member made their own 100K / 10K split of TinyStories with their own seed, used character tokenisation with their own `char_to_idx` / `idx_to_char`, trained for 10 epochs with warm-up and cosine decay, and generated text with temperature sampling and greedy decoding.

Setting	Ruchika	Akhilesh
Layers × heads × d_model	4 × 4 × 256	6 × 6 × 384
Context / dropout / tied head	256 / 0.1 / yes	256 / 0.1 / yes
Vocabulary	94 characters (min freq 5)	79 characters (min freq 50)
Optimiser, schedule	AdamW 1e-3 → 1e-4, warm-up 500 steps, cosine	AdamW 6e-4 → 6e-5, warm-up 1000 steps, cosine
Batch / seed	128 × 256 chars / 8283	64 × 256 chars / 266
Parameters	3.25 M	10.78 M

Metric (validation)	Ruchika	Akhilesh
Train / validation cross-entropy	0.608 / 0.615	**0.537 / 0.557**
Perplexity / bits per character	1.850 / 0.887	**1.745 / 0.803**
Generalisation gap (val − train)	**0.007**	0.020
Top-1 next-character accuracy	80.4%	**82.2%**
Distinct-1 / 2 / 3 (sampled)	0.298 / 0.712 / 0.877 (T 0.7)	**0.344 / 0.799 / 0.935** (T 0.8)
Repeated 4-gram rate: sampled / greedy	0.019 / 0.343	**0.009 / 0.062**
Grad-norm mean / max · loss spikes · NaNs	0.23 / 8.9 · 0 · 0	0.27 / 16.8 · 0 · 0
Training time · training chars/s	**11.0 min · 1.37 M**	33.9 min · 0.44 M
Generation chars/s · peak GPU memory	**307 · 3.05 GB**	121 · 3.48 GB

Joint analysis. Akhilesh's deeper and wider model (3.3× the parameters) reaches a clearly lower validation loss (0.557 vs 0.615 nats/char, 0.80 vs 0.89 bits per character) and higher accuracy, and its samples are more varied and repeat less. Ruchika's smaller model is about 3× faster to train and 2.5× faster to generate, and overfits slightly less (gap 0.007 vs 0.020). Both models were still improving at epoch 10. Neither run had loss spikes or NaN losses. Both members found the same three failure types in their samples: **repetition / looping under greedy decoding** (much stronger in the smaller model: repeated-4-gram rate 0.34 vs 0.06), **loss of coherence** (characters swapping names or merging into one role), and **hallucination or broken words** (invented words at high temperature; facts that contradict the prompt). Both trace these to character-level tokens, a 256-character context and the decoding temperature. Larger capacity reduces repetition but does not solve entity tracking. *Details:* `task1_llm/<member>/results.md`, `failure_analysis.md`, `metrics_report.csv`, `outputs/samples.txt`.

Task 2 — Yelp Polarity sentiment classification (embeddings from scratch)

Both members cleaned the data, analysed lengths and class balance (perfectly balanced), kept negation words when removing stopwords, trained word embeddings from scratch, and evaluated on the **official 38,000-review test set**.

	Ruchika	Akhilesh
Training data	200K-review subsample (+20K validation), seed 8283	120K-review subsample (108K train / 12K val), seed 266
Text normalisation	WordNet **lemmatisation**; NLTK stopwords minus negation/contrast words	Porter **stemming**; scikit-learn stopwords minus negation words
Vocabulary / truncation	40K words, first + last 128 tokens	30K words, first 256 tokens
Baseline	EmbBag: mean of embeddings (d 100) → **linear**	Bag of words: mean of embeddings (d 128) → **128-unit ReLU MLP**
Experimental 1	TextCNN, widths 3/4/5 × 128, dropout 0.5, 4 epochs	CNN, widths 3/4/5 × 128, dropout 0.5, 8 epochs
Experimental 2	BiLSTM (128) + **additive attention pooling**	BiLSTM (128) + **max pooling**

How the two sets differ: the baselines and the BiLSTMs differ in architecture, and every pair differs in preprocessing, data, vocabulary, truncation and training length. Both Experimental 1 models follow the standard TextCNN design (Kim, 2014). They were built independently and differ only in data, preprocessing and training length; in hindsight one of us should have chosen a different architecture for this slot.

Test set (38,000)	R · EmbBag	R · TextCNN	R · BiLSTM+attn	A · BoW	A · CNN	A · BiLSTM
Accuracy (= macro-F1)	0.9289	0.9343	**0.9425**	0.9229	0.9246	0.9309
95% CI (accuracy)	[.9264, .9312]	[.9317, .9368]	**[.9402, .9449]**	[.9202, .9253]	[.9222, .9271]	[.9284, .9335]
MCC	0.858	0.869	**0.885**	0.846	0.849	0.862
ROC-AUC / PR-AUC	.978 / .978	.983 / .984	**.987 / .987**	.976 / .976	.978 / .979	.980 / .981
Brier / ECE	.054 / .011	.048 / **.007**	**.044** / .021	.058 / **.005**	.056 / .009	.052 / .011
McNemar p vs own baseline	—	4.5e-6	5.3e-29	—	0.13	3.6e-12
Parameters	3.95 M	5.25 M	5.38 M	3.46 M	3.64 M	3.71 M
Training time	4.5 s	13.9 s	146.8 s	20.3 s	33.0 s	173.9 s
Peak GPU memory	0.30 GB	0.66 GB	0.74 GB	0.35 GB	0.62 GB	0.86 GB

Joint analysis.

- **The same ranking for both members:** bag of embeddings < CNN < BiLSTM. Reading word order helps, and the BiLSTM is significantly better than each member's baseline (McNemar p < 1e-11). Akhilesh's CNN was *not* significantly better than his baseline (p = 0.13), while Ruchika's was (p = 4.5e-6).
- **Ruchika's models are about 0.6–1.2 points more accurate at every level.** The likely reasons are the larger training subset (200K vs 120K), lemmatisation, which keeps word forms more readable than stemming, keeping contrast words ("but", "very", "only") that her preprocessing deliberately retained, and attention pooling, which focuses on the decisive sentences. Akhilesh's own testable fix ("stop removing contrast and intensity words"; try attention pooling) points to exactly these differences.
- **Calibration and accuracy trade off for both members.** The most accurate model (BiLSTM) is also the least calibrated (ECE 1.1–2.1%), while the simplest model is the best calibrated (Akhilesh's BoW ECE 0.5%). Temperature scaling would be the fix.
- **The hardest cases are the same for both:** reviews with negation or contrast, and reviews whose opinion changes inside the text (mixed sentiment, edited "it used to be good" reviews). Both error reviews also found **label noise**: reviews whose text clearly contradicts their star rating. These cases put a ceiling on the achievable accuracy.
- **Cost:** the BiLSTMs are 8–33× slower to train than the baselines for a gain of 0.8–1.4 points, and the CNNs are the best accuracy-per-second compromise.

Details: `task2_sentiment/<member>/results.md`, `failure_analysis.md` (20-error reviews with testable fixes), `metrics_report.csv`. The slice definitions differ slightly between members (different length buckets), so per-slice numbers are compared only qualitatively.

Task 3 — CycleGAN Monet ↔ photo and Kaggle

Both members implemented a CycleGAN from scratch (2 ResNet-9 generators, 2 70×70 PatchGAN discriminators, LSGAN, cycle and identity losses, image pool of 50; 28.3M parameters) and trained it on the 300 Monet paintings and the photos (A = Monet, B = photo).

Setting	Ruchika (final model `runs/final200_noid`)	Akhilesh
Upsampling	transposed convolutions	**nearest-neighbour resize + convolution**
λ_cyc / λ_id	10 / **0** (identity loss removed)	10 / **0.3**
Training	**200 epochs** × 1,000 steps (100 constant + 100 decay), fp16	30 epochs × 1,000 steps (15 + 15), fp32
Held-out data	500 photos never used in training	trained on all photos
Experiments	8 runs (training length, identity loss, DiffAugment, cycle weight, discriminator size, EMA)	1 run

Metric	Ruchika (final200_noid)	Akhilesh
Official script FID / MiFID (avg. of both directions, 300 images per side)	**101.10 / 0.414**	— (own script, see below)
photo → Monet FID / KID	91.1 / 0.0115 (500 held-out photos)	93.5 / 0.0168 (all 7,038 photos)
Monet → photo FID	93.9	113.6
photo → Monet precision / density	**0.526 / 0.454**	0.339 / 0.286
Content cosine (photo → Monet)	**0.775**	0.742
Cycle L1 (photo cycle)	**0.103**	0.146
NaN steps	0	0 of 30,000
Training time	~6.3 h (GPU shared with another run); main 30-epoch run 37 min	56.6 min
Kaggle score	**−50.7560**	−51.9920

Different evaluation setups: Ruchika's extra metrics use her shared metrics module on 500 held-out photos; Akhilesh's use his own module on all photos. The numbers are therefore close but not strictly comparable. Ruchika's Kaggle values come from the ISA's `Part3_Evaluation_Script.ipynb` run unchanged. Akhilesh's corrected submission averages both directions as the script does, but was computed with his own evaluation code on all images instead of the official 300-image setup, so it may differ slightly from the official script's output. His earlier entry (−46.99) used only the photo → Monet FID and has been replaced. The team selected −50.7560, −51.9920 and −52.5597 as its final Kaggle submissions.

Joint analysis.

- **Training length is the strongest lever.** Ruchika's series: 30 → 100 → 200 epochs gave official FID 123.5 → 104.7 → 101.1 (with the identity change in the last step). At the same 30 epochs, Akhilesh's model was better than Ruchika's 30-epoch run (photo → Monet FID 93.5 vs 125.0), which points to his **resize-convolution upsampling**. It avoids the checkerboard texture that Ruchika's transposed-convolution model produces on flat skies (her failure 1), and his λ_id of 0.3 gives a softer colour constraint.
- **The identity loss is a trade-off for both of us.** Removing it improved Ruchika's photo → Monet direction (FID 93.8 → 91.1) but hurt Monet → photo (88.2 → 93.9); Akhilesh lowered it to 0.3 for stronger style and observed colour drift.
- **Shared failure modes:** both models turn **night scenes into daylight** and add **speckle or grid texture to large flat areas** (skies, water), and both trace this to the narrow Monet domain (300 daytime paintings, almost no night scenes). Akhilesh also found grass texture painted over sand, ice and rock. Ruchika found invented sunsets and dissolving buildings in Monet → photo.
- **Stability:** neither member had NaN steps. In both runs the Monet discriminator D_A grew strong over time (Ruchika: D_A outputs ≈ 0.93 real / 0.07 fake around step 75K with four brief spikes; Akhilesh: generator gradient norms doubled in the last five epochs). Ruchika's 4-layer-discriminator run shows the extreme case: D_A memorised the 300 paintings (loss ≈ 0.001) and the generator **collapsed**, hiding the photo in an invisible noise pattern (photo → Monet FID 216.7, precision 0.03) while still reconstructing it almost perfectly. A low cycle loss alone does not prove good translations.
- **Cycle consistency is verified for both:** reconstruction error is far below the input-to-translation distance (Ruchika 0.103 vs 0.29; Akhilesh 0.146 / 0.142 in the two directions).
- **Shortcomings (both):** single seeds; FID on few images is noisy and biased upwards; Monet → photo is evaluated on the 300 training paintings; and the Monet discriminator overfits easily. Next steps both of us identified: spectral normalisation or a weaker discriminator, resize-convolution upsampling (for Ruchika), longer training (for Akhilesh), and a small identity weight as a compromise.

Human audit (blinded, 2 raters). Both members' final photo → Monet models were rated on 60 items: Ruchika's 30 team-fixed audit photos (model `final200_noid`) and Akhilesh's 30 audit photos, each shown as *original photo* | *Monet-style output*. The items were shuffled and anonymised (Q01–Q60) with a sealed key, and the two raters (Ruchika and Akhilesh) scored each item independently from 1 to 5 for **style** (does it look like a Monet painting), **content** (is the scene preserved) and **artifacts** (5 = none visible).

Mean score (both raters, 1–5)	Ruchika (final200_noid)	Akhilesh	Mann–Whitney p
Style	**4.30** ± 0.52	4.18 ± 0.52	0.37
Content	**4.10** ± 0.53	4.05 ± 0.50	0.77
Artifacts (higher = fewer)	**4.30** ± 0.43	4.02 ± 0.46	**0.023**
Overall	**4.23**	4.08	

Inter-rater agreement (60 items)	Style	Content	Artifacts	All criteria
Cohen's κ (quadratic-weighted)	0.06	0.14	0.42	0.18
Cohen's κ (unweighted)	0.03	0.20	0.28	—
Exact agreement	40%	52%	65%	52%
Agreement within 1 point	88%	90%	100%	93%

Both raters judged both models favourably (means 4.0–4.3), and the two models are rated similarly for style and content. The only significant difference is in **artifacts**, where Ruchika's model scored higher (4.30 vs 4.02, p = 0.023), consistent with its much longer training (200 vs 30 epochs). Agreement is *moderate* for artifacts (κ = 0.42) but only *slight* for style and content (κ = 0.06 and 0.14). The raters almost never differed by more than one point (93% within 1), but all scores fell in the narrow 3–5 range, and κ is known to be low when ratings are concentrated in a few categories. Judging "how Monet-like" an image looks is also more subjective than spotting

glitches. **Limitations:** the two models were rated on different input photos, so the comparison is across inputs rather than paired; both raters are the models' authors (blinding reduces but does not remove this bias); and 30 items per model is a small sample. Files: `task3_gan/audit/` (rating sheets, sealed key, results). *Details:* `task3_gan/<member>/results.md`, `failure_analysis.md`, `full_metrics_report.csv`, `submission.csv`; Ruchika's experiment series in `task3_gan/ruchika_chaurasia/runs/`.

Reproducibility and evidence

- Executed notebooks with outputs, configs, checkpoints and outputs are in each member's folder; the large non-submitted checkpoints and the datasets are linked in the README (Google Drive).
- Unedited, timestamped raw logs and run manifests (hardware, package versions, configuration, checkpoint SHA-256 IDs): `reproducibility/` (Ruchika) and `task*/akhilesh_tandur/logs/` (Akhilesh). One-command smoke test: `bash reproducibility/smoke_test.sh`.
- Checkpoint IDs of Ruchika's reported models: Task 1 `bceaa3cfc2f2`; Task 2 `4cf09316ac72`, `977844e053f5`, `ff1c3543fe50`; Task 3 `aec6800f8b2e` (main), `cb67fbc511ff` (long100), `bcbcfc109a7e` (final200_noid).
- AI use is disclosed in `AI_USE.md`.

References

1. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.
2. Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*.
3. Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. *ICCV*.
4. Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *NeurIPS* (Yelp Polarity).
5. Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *EMNLP*.
6. Joulin, A., et al. (2017). Bag of Tricks for Efficient Text Classification. *EACL*.
7. Heusel, M., et al. (2017). GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium (FID). *NeurIPS*.
8. Bińkowski, M., et al. (2018). Demystifying MMD GANs (KID). *ICLR*.
9. Kynkäänniemi, T., et al. (2019). Improved Precision and Recall Metric for Assessing Generative Models. *NeurIPS*.
10. Naeem, M. F., et al. (2020). Reliable Fidelity and Diversity Metrics for Generative Models. *ICML*.
11. Zhang, R., et al. (2018). The Unreasonable Effectiveness of Deep Features as a Perceptual Metric (LPIPS). *CVPR*.
12. Mao, X., et al. (2017). Least Squares Generative Adversarial Networks. *ICCV*.
13. Odena, A., Dumoulin, V., & Olah, C. (2016). Deconvolution and Checkerboard Artifacts. *Distill*.
14. Zhao, S., et al. (2020). Differentiable Augmentation for Data-Efficient GAN Training. *NeurIPS*.
15. Yazıcı, Y., et al. (2019). The Unusual Effectiveness of Averaging in GAN Training. *ICLR*.
16. Chu, C., Zhmoginov, A., & Sandler, M. (2017). CycleGAN, a Master of Steganography. *arXiv:1712.02950*.
17. Guo, C., et al. (2017). On Calibration of Modern Neural Networks. *ICML*.
18. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions. *Psychometrika*.